

Resumen-Tema-2-para-examen.pdf



dcardenas11



Inteligencia Artificial



2º Grado en Ingeniería Informática



Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación
Universidad de Granada

Formamos
talento para un futuro
Sostenible



MÁSTER EN

**Big Data &
Business Analytics**

EOI Escuela de
organización
industrial

saber más

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

perdo
espacio



Necesito
concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH

Tema 2. Agentes

Tema 2. Agentes

- Agentes inteligentes
 - Introducción
 - Sistemas basados en agentes
 - Sistemas multi-agente
 - Características de un SMA
 - Arquitecturas de Agentes
 - Arquitecturas deliberativas
 - Arquitecturas Reactivas
 - Arquitecturas Híbridas
 - Agentes reactivos
 - Representaciones del mundo
 - Diseño de un agente reactivo
 - Función del agente
 - Percepción
 - Acción
 - Sistemas de producción
 - Agentes reactivos con memoria
 - Implementación de la memoria con representaciones icónicas
 - Arquitecturas alternativas de agentes reactivos
 - Arquitecturas basadas en Redes de unidades lógicas con umbral
 - Unidad lógica con umbral
 - Arquitectura de subsunción
 - Limitaciones de un agente reactivo
- Agente deliberativo
 - Diseño de un agente deliberativo
 - Componentes en la descripción de un problema
 - La búsqueda en un espacio de estados
 - Grafo de estados
 - Búsqueda
 - Sistemas de búsqueda y estrategias
 - Procedimiento general de búsqueda
 - Estrategia de control
 - Estrategias irrevocables
 - Estrategias tentativas
 - Estrategia retroactiva (Backtracking)
 - Búsqueda en grafos vs árboles
 - Medidas del comportamiento de un sistema de búsqueda

Agentes inteligentes

Introducción

Podemos definir la IA como el subcampo de la Informática dedicado a la construcción de agentes que exhiben aspectos del comportamiento inteligente, por tanto los agentes permiten dar una nueva forma de mostrar la Inteligencia Artificial.

WUOLAH

Un **Agente Inteligente** es un sistema de ordenador, *situado* en algún entorno, que es capaz de realizar acciones de forma *autónoma* y que es *flexible* para lograr los objetivos planteados. Por tanto, tenemos tres aspectos que hay que tener en cuenta:

1. Situación: el agente recibe entradas sensoriales de un entorno en donde está situado y realiza acciones que cambian dicho entorno.
2. Autonomía: el sistema es capaz de actuar sin la intervención directa de los humanos y tiene control sobre sus propias acciones y estado interno.
3. Flexibilidad, donde podemos observar cuatro características deseables del agente:
 - Reactivo: el agente debe percibir el entorno y responder de una forma temporal a los cambios que ocurren en dicho entorno.
 - Deliberativo: el agente percibe los cambios en el entorno y conoce las consecuencias de sus acciones en el entorno.
 - Pro-activo: los agentes no deben simplemente actuar en respuesta a su entorno, deben de ser capaces de exhibir comportamientos dirigidos a lograr objetivos que sean oportunos, y tomar la iniciativa cuando sea apropiado.
 - Social: los agentes deben de ser capaces de interactuar, cuando sea apropiado, con otros agentes artificiales o humanos para completar su propio proceso de resolución del problema y ayudar a otros con sus actividades.

Sistemas basados en agentes

Un **Sistema Basado en Agentes** será un sistema en el que la abstracción clave utilizada es precisamente la de agente. Podemos particularizar esta definición con los **Sistemas multi-agente**, que son sistemas diseñados e implementados con varios agentes interactuando.

Sistemas multi-agente

Los sistemas multi-agente son interesantes para representar problemas que tienen

- Múltiples formas de ser resueltos
- Múltiples perspectivas
- Múltiples entidades para resolver el problema

Un sistema multi-agente, por tanto, necesita controlar la interacción entre agentes, para eso definimos:

- **Cooperación:** trabajar juntos para resolver algo
- **Coordinación:** organizar una actividad para evitar las interacciones perjudiciales y explotar las beneficiosas.
- **Negociación:** llegar a un acuerdo que sea aceptable por todas las partes implicadas.

La Inteligencia Artificial Distribuida se encarga de la resolución de problemas distribuida, utilizando Sistemas Multi-Agente (igual es un triple pero tiene sentido).

Por tanto podemos dar una definición más exacta de un **Sistema Multi-Agente (SMA)**: SMA: una red más o menos unida de resolutores de problemas que trabajan conjuntamente para resolver problemas que están más allá de las capacidades individuales o del conocimiento de cada resolutor del problema, donde resolutor=agente autónomo de naturaleza heterogénea.

¿VAS A ESTUDIAR EN GRANADA?

Descubre la **Residencia Universitaria Xior** perfecta para ti.

📍 XIOR GRANADA



Habitaciones y pisos para
estudiantes en Granada
en **Xior Student Housing**.

Más info



Xior, Tu Hogar Lejos de Casa

XIOR
STUDENT HOUSING

Inteligencia Artificial



Comparte estos flyers en tu clase y consigue más dinero y recompensas



Banco de apuntes de la

WUOLAH

- 1** Imprime esta hoja
- 2** Recorta por la mitad
- 3** Coloca en un lugar visible para que tus compis puedan escanar y acceder a apuntes

- 4** Llévate dinero por cada descarga de los documentos descargados a través de tu QR



Características de un SMA

- Cada agente tiene información incompleta, o no todas las capacidades para resolver el problema, así cada agente tiene un punto de vista limitado.
- No hay un sistema de control global.
- Los datos no están centralizados.
- La computación es asíncrona.
- Cooperación y negociación.

Arquitecturas de Agentes

Arquitecturas deliberativas

Antes de comenzar, definimos:

- **Sistema de símbolos físicos:** un conjunto de entidades físicas (símbolos) que pueden combinarse para formar estructuras, y que es capaz de ejecutar procesos que operan con dichos símbolos de acuerdo a conjuntos de instrucciones codificadas simbólicamente.
 - La **hipótesis de sistema de símbolos físicos** dice que tales sistemas son capaces de generar acciones inteligentes.
- **Agente deliberativo:** aquel que contiene un modelo simbólico del mundo explícitamente representado, y cuyas decisiones se realizan a través de un razonamiento lógico basado en emparejamientos de patrones y manipulaciones simbólicas.

Las arquitecturas deliberativas serán, por tanto, las que incorporen un agente deliberativo. Este tipo de arquitecturas presentan dos problemas:

- Trasladar en un tiempo razonable (para que sea útil) el mundo real en una descripción simbólica precisa y adecuada que pueda utilizar el agente.
 - Como lo transformamos.
- Representar simbólicamente la información acerca de entidades y procesos complejos del mundo real, y como conseguir que los agentes razonen con esta información para que los resultados sean útiles.
 - Que cogemos del mundo real.

Arquitecturas Reactivas

Una **arquitectura reactiva** es aquella que no incluye ninguna clase de modelo centralizado de representación simbólica del mundo, y no hace uso de razonamiento complejo:

- El comportamiento inteligente puede ser generado sin una representación explícita de la clase que la IA simbólica propone.
- La inteligencia es una propiedad emergente de ciertos sistemas complejos.

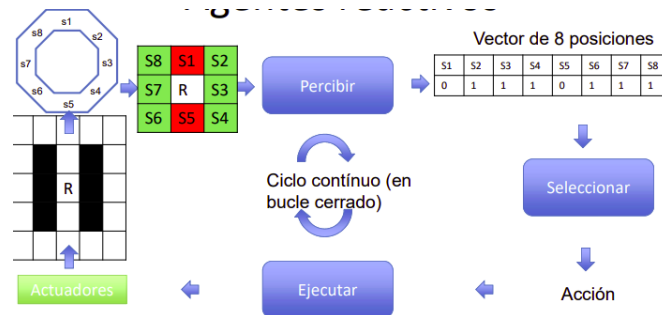
En estas arquitecturas, la inteligencia “real” está situada en el mundo, y no en sistemas incorpóreos tales como la demostración de teoremas o los sistemas expertos, y el comportamiento “inteligente” surge como el resultado de la interacción del agente con su entorno. La inteligencia está “en el ojo de espectador” no es una propiedad innata ni aislada.

Arquitecturas Híbridas

Tienen una estructura vertical, por capas. En la capa más baja se produce una entrada sensorial, que se procesa en las siguientes capas, y produce como salida una acción.

Agentes reactivos

Un Agente reactivo es el que a través de una entrada sensorial, que se procesa en **Percibir**, se produce una representación interna con alguna estructura de datos. Con esa representación del mundo real se toma una decisión que será la acción ejecutada. Este bucle se repite pues la acción cambiará la percepción del mundo real.



Representaciones del mundo

Podemos tener dos tipos de representaciones:

- **Modelo icónico**, es una estructura de datos que muestra un reflejo lo más fiel posible de lo que se observa.
 - En este modelo, las entradas sensoriales se utilizan en primera instancia para actualizar la representación icónica. Una vez actualizada, se realizan cálculos para extraer las características que necesita el agente para tomar la decisión de la acción.
 - Existen varios tipos de acciones, las que modifican la representación icónica o las que modifican el entorno.
 - Las características derivadas de un modelo icónico deben representar el entorno de una forma adecuada al tipo de acciones que se deben tomar.
- **Modelo basado en características**, se centran en definir los objetos del mundo y asociar características/atributos a cada uno de ellos (propiedades y relaciones).
 - En este modelo, se establece un vector de características, que representará al mundo real, basado en las entradas sensoriales.
 - Además, normalmente el vector de características calculado depende del vector de características anterior.
 - El problema que presentan es que no se suelen obtener representaciones precisas del entorno mediante estos vectores.

Diseño de un agente reactivo

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

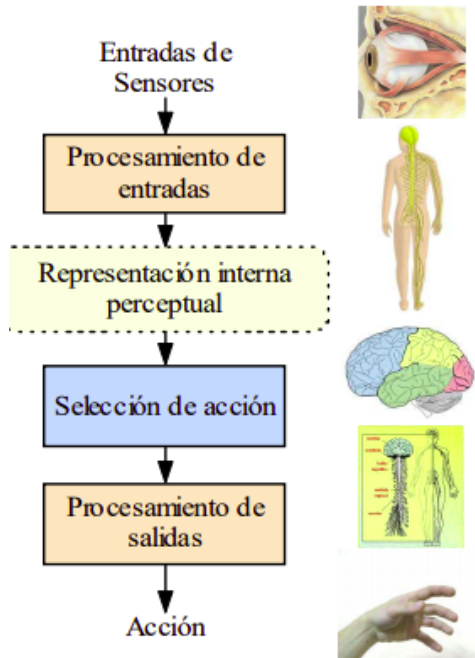
perdo
espacio



Necesito
concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH



Dentro del ciclo que hemos visto antes, describimos el proceso de diseño de un agente reactivo:

- Percepción y Acción:
 - El agente reactivo percibe su entorno a través de sensores.
 - Procesa la información percibida y hace una representación interna de la misma.
 - Escoge una acción, entre las posibles, considerando la información percibida.
 - Transforma la acción en señales para los actuadores y la realiza.

Por tanto es necesario considerar los siguientes elementos:

- **Descripción del entorno**, se describe donde se situará el robot.
- **Información sensorial**, de qué sensores dispone.
- **Especificación del comportamiento**, establecer el objetivo.
- **Repertorio de acciones**, qué acciones puede tomar.
- **Restricciones adicionales**

El trabajo del diseñador es, por tanto **determinar la función del agente**, es decir, desarrollar una función definida sobre las entradas sensoriales que seleccione la acción apropiada en cada momento para llevar a cabo con éxito la tarea del robot.

Función del agente

Para diseñar la función del agente establecemos dos fases:

- La primera fase es el procesamiento perceptual, mediante el cual se establece el vector de características de este paso. Se procesan las entradas sensoriales y se dan los valores a cada característica escogida, con su significado asociado.
 - El **vector de características** almacena valores binarios representando características del mundo que nos interesan. No solo información sensorial. Es decir, podemos sacar alguna conclusión procesando las entradas sensoriales.
- La segunda fase es el cálculo de la acción, mediante la cual se estudia el vector de características y se escoge la acción a llevar a cabo.

WUOLAH

Percepción

La fase de percepción, mediante la cual se dan los valores al vector de características. Hay que tener en cuenta que en muchas de las tareas que existen en el mundo real, el procesamiento perceptual puede producir a veces información errónea o ambigua que puede llevar a una selección inadecuada de la acción.

Se establecen características adicionales a las entradas sensoriales.

Acción

En esta fase, a partir de las características se define una política de actuación. Es necesario definir los criterios de selección de acción.

Sistemas de producción

Los sistemas de producción proporcionan una representación adecuada para las funciones de selección de acción. Está formado por un conjunto de reglas llamadas **producciones**.

Cada regla se escribe de la siguiente manera: $c_i \rightarrow a_i$, donde c es la condición y a es la acción.

La condición de una regla puede ser una función booleana definida sobre el vector de características, y suele ser una conjunción de literales booleanos.

La selección de la acción se realiza de la siguiente manera: se comienza con la primera regla, y se busca, siguiendo el orden establecido en el conjunto de reglas, una de ellas en la que la evaluación de su condición proporcione el valor 1, y se selecciona su parte de acción.

La parte de acción puede ser una acción primitiva, una llamada a otro sistema de producción o un conjunto de acciones que tienen que ser ejecutadas simultáneamente. En la mayor parte de los casos, la última regla del conjunto tiene como condición el valor 1, y, por tanto, si ninguna de las condiciones de las reglas anteriores toma el valor 1, se ejecuta por defecto la acción asociada a esta última regla. A medida que se van ejecutando las acciones, se van modificando las entradas sensoriales, así como los valores de las características derivados a partir de aquellas.

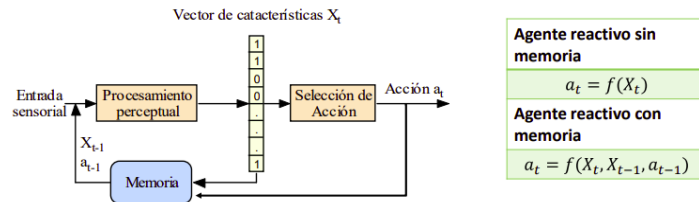
Agentes reactivos con memoria

Los agentes reactivos vistos hasta ahora solamente tienen en cuenta las entradas sensoriales en cada paso, por lo que están muy limitados a su sistema sensorial, que ya sabemos que no nos da información completa. Para arreglar este fallo surgen los **agentes reactivos con memoria**, que mejoran la precisión teniendo en cuenta la historia sensorial previa.

Consideraremos que la representación de un estado en el instante t es función de:

- Las entradas sensoriales en el instante t
- La representación del estado en el instante anterior $t-1$, es decir, el vector de características.
- La acción seleccionada en el instante anterior $t-1$.

Por tanto, la decisión de qué acción ejecutar en el instante t depende de esta nueva representación del estado.



Implementación de la memoria con representaciones icónicas

Además del vector de estados, el robot podría utilizar otras estructuras de datos que representen el mundo real: matriz que almacene el mapa con las casillas libres u ocupadas en el momento en el que se percibieron.

Arquitecturas alternativas de agentes reactivos

Arquitecturas basadas en Redes de unidades lógicas con umbral

Ya hemos visto que se pueden construir programas que implementen funciones booleanas y sistemas de producción de una forma sencilla. Alternativamente, los sistemas de producción se pueden implementar directamente por medio de **circuitos electrónicos**. El circuito puede tener como entradas las propias señales sensoriales. Utilizando circuitos lógicos, una función booleana se puede implementar mediante una red de puertas lógicas (ANO, NANO, OR, etc.).

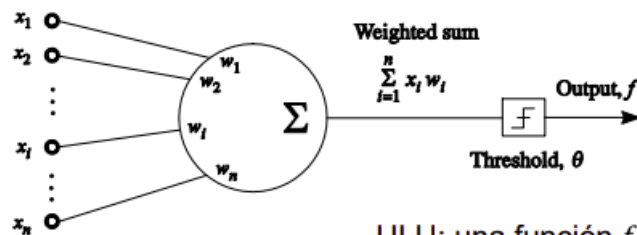
Un tipo muy común de circuito lo constituyen las redes formadas por elementos con umbral o por algún

otro tipo de elemento que aplique una función no lineal sobre una suma ponderada de sus entradas. Un ejemplo de este tipo de elementos es la Unidad Lógica con Umbral (ULU).

Unidad lógica con umbral

Esta unidad realiza una suma ponderada de las entradas, compara esta suma con un umbral y, si supera el umbral, produce como salida un 1; en cualquier otro caso, produce un 0.

Es decir:



$$f = 1 \text{ if } \sum_{i=1}^n x_i w_i \geq \theta$$

$$= 0 \text{ otherwise}$$

© 1998 Morgan Kaufmann Publishers

ULU: una función $f: \mathbb{R}^n \rightarrow \{0,1\}$ con n entradas y la salida en $\{0,1\}$

Cálculo:

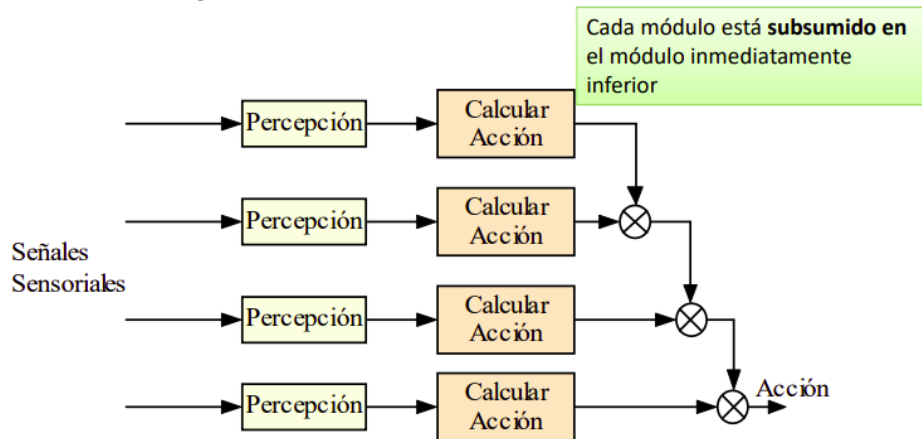
- Combinación lineal de las entradas con unos pesos previamente definidos
- La salida de la ULU es 1 o 0, dependiendo de si la combinación lineal supera o no el umbral previamente definido.

Aunque a veces nos valga una ULU para solucionar el problema de selección de acción, muchas veces será necesaria una Red de ULU, también llamado **Red Neuronal**. De esta manera diferentes ULUs se pueden combinar para obtener las mismas salidas que con en el Sistema de producción completo.

Las Redes neuronales son útiles cuando el comportamiento requerido es difícil de describir manualmente. En estos casos se recurre al aprendizaje.

Arquitectura de subsunción

La arquitectura de subsunción consiste en agrupar **módulos de comportamiento**. Cada módulo de comportamiento tiene una acción asociada, recibe la percepción directamente y comprueba una condición. Si esta se cumple, el módulo devuelve la acción a realizar.



En caso de conflicto entre dos módulos, si las dos condiciones se activan **tiene prioridad el módulo inferior**.

Limitaciones de un agente reactivo

En un agente reactivo, podemos complicar el modelo de memoria interna todo lo que queramos. El agente puede recordar lo que ha hecho, si modificamos el modelo de memoria, pero **no tiene capacidad para conocer de forma anticipada cómo se transforma el mundo mediante la ejecución de sus acciones**.

Además, no tiene capacidad para conocer cuál es su meta u objetivo para tomar una decisión correcta entre varias alternativas.

Agente deliberativo

Antes de comenzar a tratar el diseño de un agente deliberativo, enunciaremos alguna característica:

- El agente dispone de un **modelo del mundo en el que habita**:
 - Por ejemplo un mapa del entorno y/o qué objetos hay en el entorno y qué propiedades tienen esos objetos.
- El agente dispone de un **modelo de los efectos de sus acciones** sobre el mundo
 - P.e. sabe anticipadamente en qué casilla se encontrará cuando aplica una acción.
- El agente es capaz de **razonar sobre esos modelos** para decidir que hacer para conseguir un objetivo.
 - P.e., si la hormiga sabe que hay comida en (4,4), y conoce las consecuencias de sus acciones, en cada casilla puede tomar una decisión correcta para alcanzar su objetivo: "ir a la casilla (4,4) para comer".

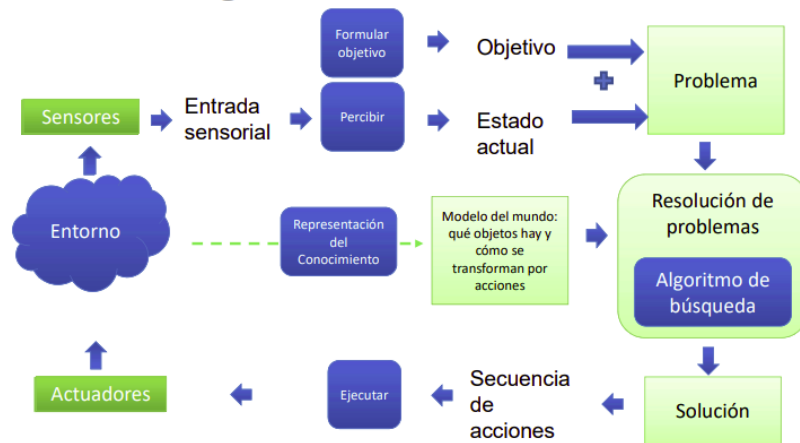
Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

Diseño de un agente deliberativo

La clave en este tema es que el agente con varias opciones inmediatas puede usar esa información para proyectar y considerar etapas subsecuentes sobre un recorrido hipotético, tratando de encontrar un camino que le lleve finalmente hasta su objetivo. En otras palabras, analizará todas las consecuencias de sus acciones proyectando hacia el futuro las opciones inmediatas que ya conoce, hasta que encuentre un objetivo.



El diseño incorpora aspectos nuevos, que son los que se tienen en cuenta a la hora de decidir la acción:

- Estado actual, que viene de la percepción de las entradas sensoriales.
- Formulación del objetivo, que ya sabemos que es una característica de un agente deliberativo.
- Representación del conocimiento, mediante el modelo del mundo real.

Componentes en la descripción de un problema

- **Estado inicial**, se puede percibir.
- **Estado objetivo**, puede proporcionarse como entrada al agente (un humano) o bien podemos tener un agente pro-activo que sea capaz formular objetivos para poder llegar a la autonomía plena.
- **Acciones**, repertorio de acciones del agente.
 - Para cada estado particular podemos representar las acciones que se pueden ejecutar.
- **Efectos de las acciones** (o modelo de transición), una forma de especificar el estado resultante de aplicar una acción en un estado dado.
 - Para cada estado particular podemos conocer las acciones que se pueden ejecutar y su **efecto**, es decir, su estado resultante.
- **Espacio de estados**, el conjunto de estados alcanzables desde el estado inicial mediante una secuencia de acciones.
 - Está definido por:
 - El estado inicial
 - El estado objetivo
 - Modelo de transición

- El espacio de estados tiene la forma de un **grafo dirigido** en el que cada nodo representa una casilla y cada arco una acción de movimiento entre casillas. Por tanto, un **camino en el espacio de estados** es una secuencia de estados conectados por una secuencia de acciones.
- **Comprobación de objetivo**, una función para comprobar que un estado dado es objetivo.
 - A veces hay un conjunto explícito de estados objetivo, que puede contener un único estado, y esta función simplemente comprueba que el estado dado es uno de ellos.
- **Coste del camino**, una función que asigna un coste numérico a cada camino.
 - Se usa para obtener una medida del rendimiento del agente. Es la suma de los costes individuales de cada acción del camino.

Con estos elementos podemos definir un problema y podemos representarlo todo en una única estructura que proporcionamos a un algoritmo de resolución de problemas. Diremos que una **solución a un problema** es una secuencia de acciones que empieza en el estado inicial y permite alcanzar el estado objetivo.

En los problemas en los que las acciones tengan un coste podemos hablar de **solución óptima** que consiste en el camino con el coste más bajo de entre todas las posibles soluciones.

La búsqueda en un espacio de estados

Diremos que la inteligencia artificial está formada por la representación del conocimiento y por la búsqueda. Con los componentes mencionados (Estado, Estado inicial, Estado Objetivo, Acciones, Coste, Espacio de estados) podemos representar el conocimiento a través de las acciones del agente.

Nuestro problema entonces es la búsqueda en el espacio de estados, es decir, la resolución del problema mediante proyección de las distintas acciones y búsqueda del camino solución.

Grafo de estados

Ya hemos visto que el espacio de estados tiene forma de **grafo dirigido**, esta estructura es útil para buscar secuencias de acciones que nos lleven al objetivo final.

En esta estructura, un nodo representa un estado del sistema y un arco una posible acción. La acción, aplicada al estado que representa al nodo origen, producirá el estado del nodo destino. Se denomina **grafo de estados**.

A la secuencia de acciones que lleva al agente desde un estado inicial hasta un estado destino se denomina **plan**. La búsqueda de dicha secuencia se denomina **planificación**. Podemos diferenciar dos tipos de grafos:

- Grafos explícitos: grafo que representa el espacio de estados explorado hasta el momento y que se almacena físicamente en memoria.
- Grafos implícitos: grafo que representa el espacio de estados en su totalidad, no está almacenado físicamente en memoria.

Búsqueda

El grafo explícito se va construyendo por etapas conforme vamos “descubriendo” sus nodos aplicando acciones. En el primer ejemplo hemos visto cómo el grafo se va construyendo progresivamente a partir del estado inicial.

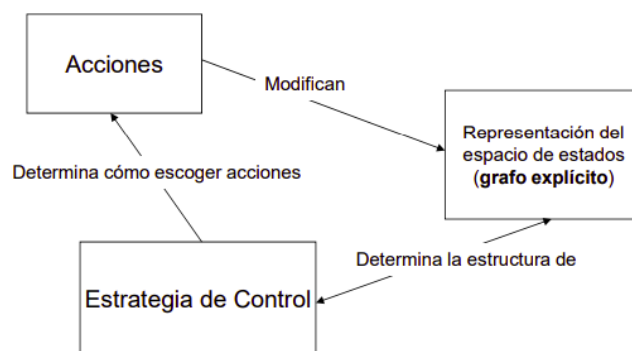
Hay problemas en los que puede ser conveniente partir del objetivo y aplicar las acciones en sentido inverso, haciendo un proceso de exploración del grafo regresivo.

(De esto hay mil ejemplos en las diapos, mirar si eso pero no rents)

Sistemas de búsqueda y estrategias

Una vez que formulamos un problema, tenemos que resolverlo, con un proceso de búsqueda que tiene que encontrar una secuencia de acciones partiendo del estado inicial como nodo raíz del grafo explícito o del grafo/árbol de búsqueda, los arcos son acciones y los nodos son estados del espacio de estados del problema, tal y como hemos dicho ya 193948 veces.

En este tipo de problemas **no hay una única estrategia de búsqueda**:



Procedimiento general de búsqueda

El procedimiento es:

1. **Inicialización** del grafo explícito con el estado inicial.
2. Hasta que el grafo contenga un nodo que cumple la condición de terminación (*comprobación de objetivo*), hacer:
 1. Seleccionar alguna acción A en el conjunto de acciones que pueda ser aplicada al grafo.
 2. Introducir en el grafo el resultado de aplicar A al grafo.

Estrategia de control

Dependiendo de cómo representemos el grafo explícito podemos usar distintas estrategias para explorarlo, tenemos:

- **Estrategias irrevocables**
- **Estrategias tentativas**
 - Retroactivas
 - Búsqueda en grafos

Estrategias irrevocables

En cada momento, el grafo explícito lo constituye un único nodo, E, que incluye la descripción completa del sistema en ese momento. En cada ciclo, entonces:

- Se selecciona una acción A.
- Se aplica sobre el estado del sistema E, para obtener el nuevo estado $E' = A(E)$.

- Se borra de memoria E y se sustituye por E'.

Se termina cuando E verifica la *condición de estado objetivo*.

Estrategias tentativas

Estrategia retroactiva (Backtracking)

En memoria sólo guardamos un hijo de cada estado; esto es, se mantiene el camino desde el estado inicial hasta el actual. El grafo explícito, por tanto, es realmente **una lista**.

El proceso para cuando hemos llegado al objetivo y no deseamos encontrar más soluciones, o bien no hay más operadores aplicables al nodo raíz del espacio de búsqueda.

En este tipo de algoritmos, podemos **retroceder**, es decir, eliminar el último nodo que hemos añadido a la lista y aplicar otro de los operadores que todavía no se ha aplicado al nodo anterior. Esta vuelta atrás, se produce cuando:

- Se ha encontrado una solución, pero deseamos encontrar otra solución alternativa.
- Se ha llegado a un límite en el nivel de profundidad explorado o el tiempo de exploración en una misma rama.
- Se ha generado un estado que ya existía en el camino.
- No existen reglas aplicables al último nodo de la lista (último nodo del grafo explícito).

Búsqueda en grafos vs árboles

Si utilizamos un grafo explícito, en memoria se guardan todos los estados (o nodos) generados hasta el momento, de forma que la búsqueda puede proseguir por cualquiera de ellos.

- **En búsqueda en árboles** se usa una lista denominada **Frontera (o Abiertos)** donde se almacenan los nodos sucesores que se han generado en etapas del algoritmo y que están pendientes de explorar.
- **En búsqueda en grafos** se usa una lista adicional llamada **Explorados (o Cerrados, o Visitados)** donde se almacenan los nodos que fueron frontera y que se han explorado (para llevar una traza de los nodos que ya se han visitado).

De esta manera, la búsqueda será:

```
function TREE-SEARCH(problem) returns a solution, or failure
  initialize the frontier using the initial state of problem
  loop do
    if the frontier is empty then return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state then return the corresponding solution
    expand the chosen node, adding the resulting nodes to the frontier

function GRAPH-SEARCH(problem) returns a solution, or failure
  initialize the frontier using the initial state of problem
  initialize the explored set to be empty
  loop do
    if the frontier is empty then return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state then return the corresponding solution
    add the node to the explored set
    expand the chosen node, adding the resulting nodes to the frontier
    only if not in the frontier or explored set
```

Para los algoritmos de búsqueda, es necesaria una infraestructura, en otras palabras, una función que implementa el modelo de transición, es decir, la forma de calcular el estado sucesor a uno dado, o de otra forma, calcular los efectos de aplicación de una acción.

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

pierdo
espacio



Necesito
concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH

```
function CHILD-NODE(problem, parent, action) returns a node
  return a node with
    STATE = problem.RESULT(parent.STATE, action),
    PARENT = parent, ACTION = action,
    PATH-COST = parent.PATH-COST + problem.STEP-COST(parent.STATE, action)
```

Medidas del comportamiento de un sistema de búsqueda

1. **Compleitud:** hay garantía de encontrar la solución si esta existe.
2. **Optimalidad:** hay garantía de encontrar la solución óptima.
3. **Complejidad en tiempo:** ¿Cuánto tiempo se requiere para encontrar la solución?
4. **Complejidad en espacio:** ¿Cuánta memoria se requiere para realizar la búsqueda?

WUOLAH